Academi-Gymraeg-Group-Project

Overview

Academi Gymraeg is a Spring Boot web application designed to support Welsh language learning. The system implements role-based access control and allows users to interact with tests and learning content based on their permissions.

Tech Stack

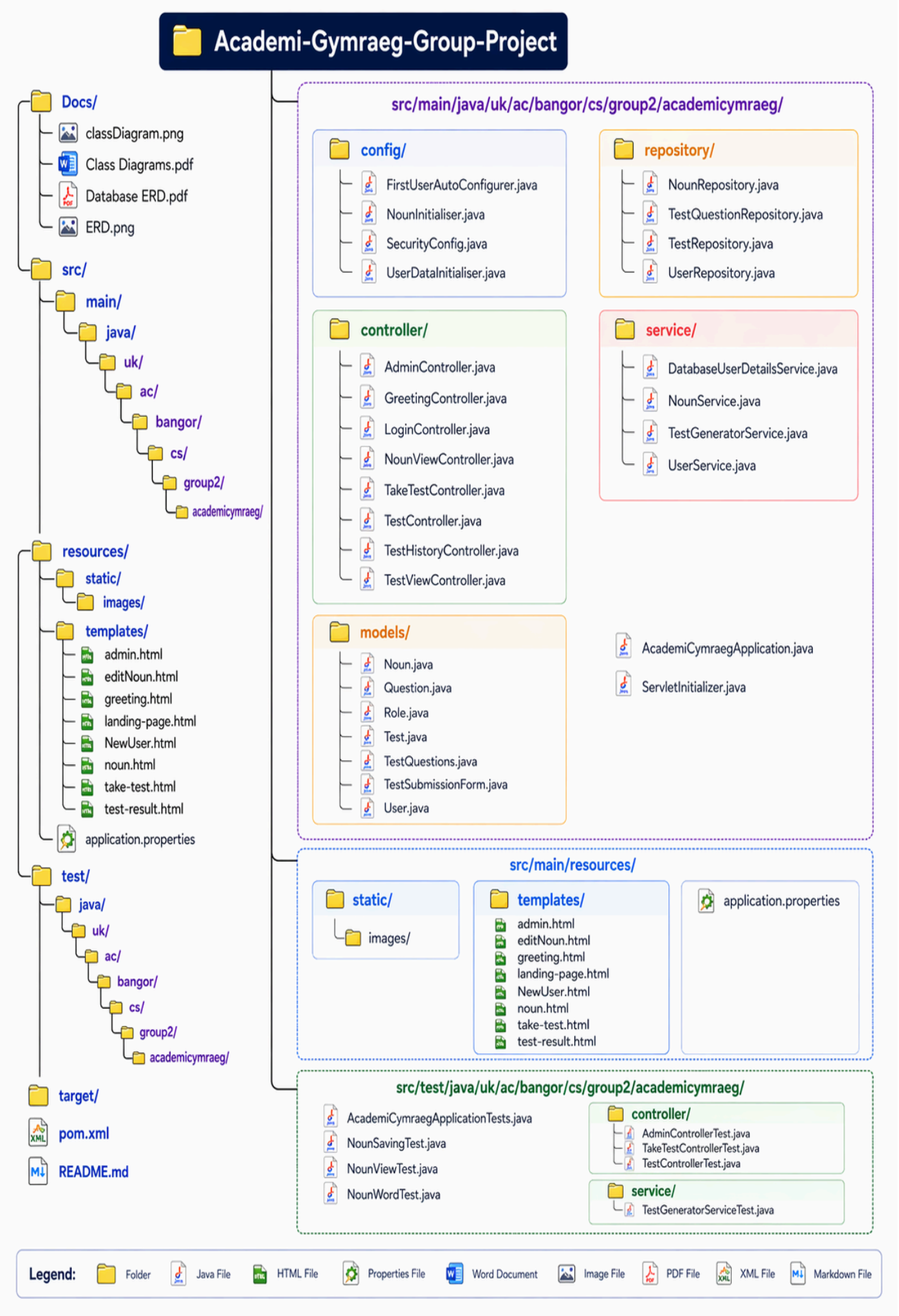
- Java 17+
- Spring Boot
- Spring Security
- Spring Data JPA
- Maven
- MySQL
- Eclipse IDE

Architecture

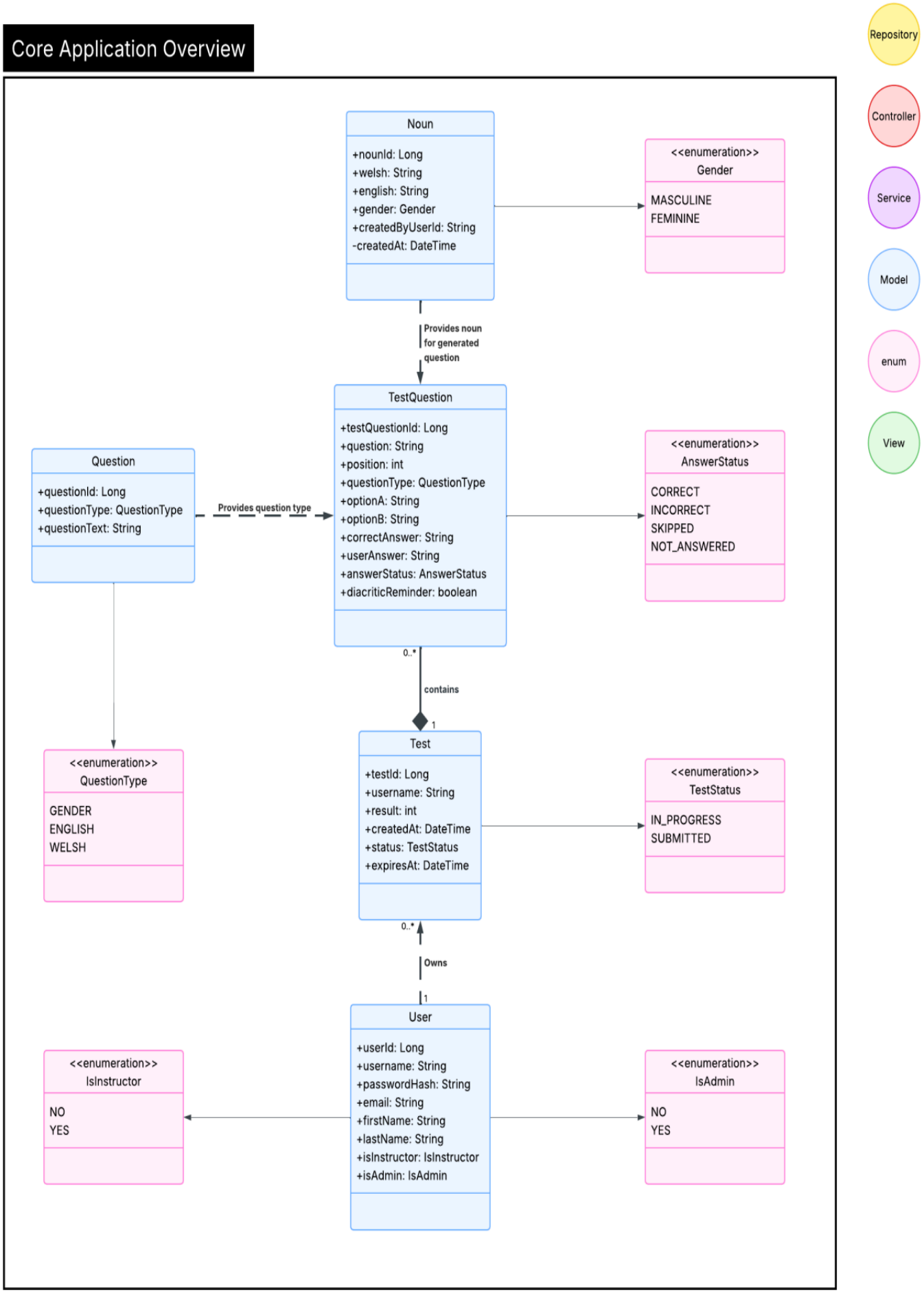
The project follows a standard layered Spring Boot structure:

config/ -> Security configuration (Spring Security) models/ -> JPA entities (e.g. User) repository/ -> Data access layer (JpaRepository interfaces) service/ -> Business logic (UserService, authentication) controller/ -> Web layer (HTTP endpoints)

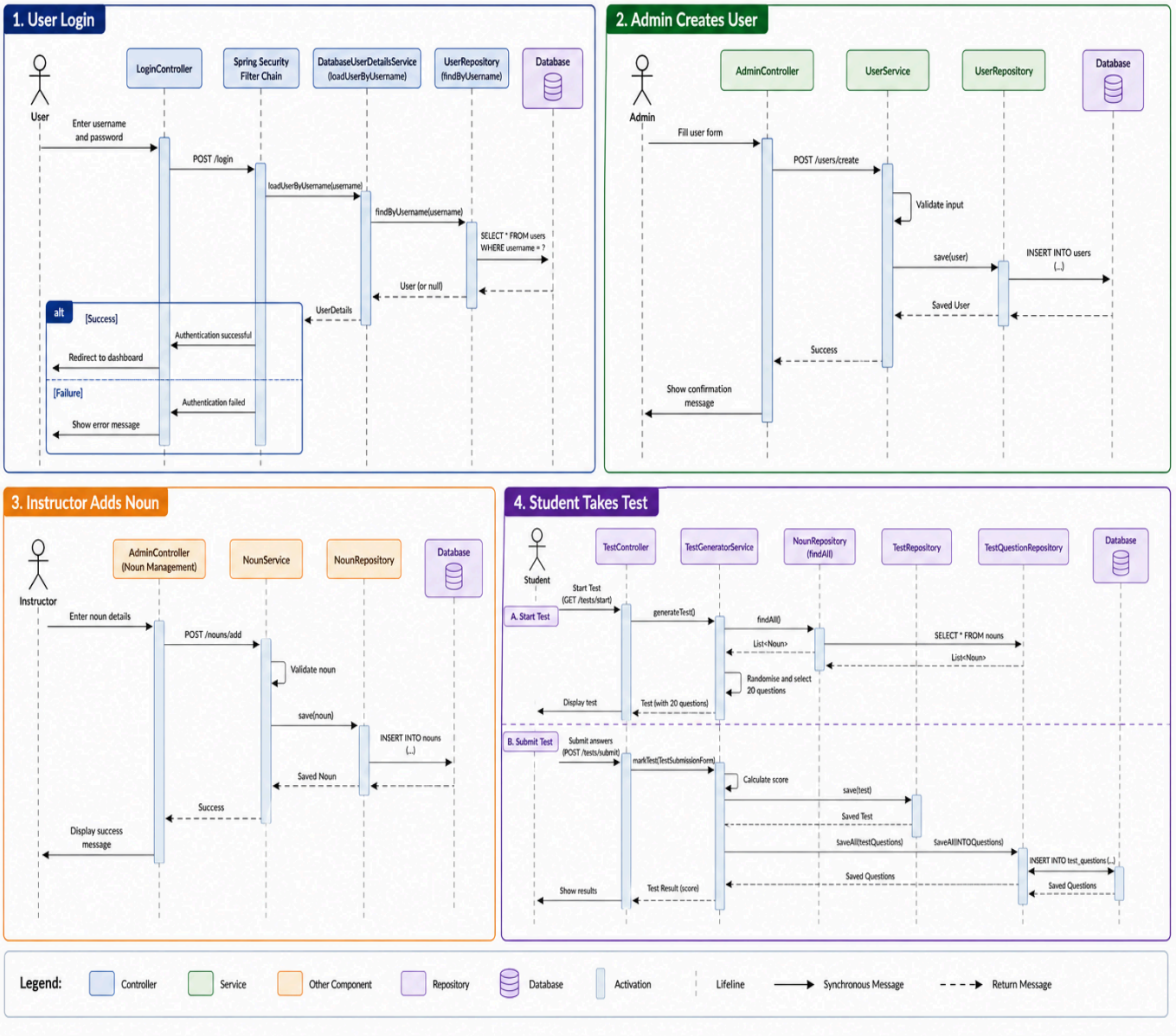
File Structure


Academi-Gymraeg-Group-Project
Docs/
classDiagram.png
Class Diagrams.pdf
Database ERD.pdf
ERD.png
src/
main/
java/
uk/
ac/
bangor/
cs/
group2/
academicymraeg/
resources/
static/
images/
templates/
admin.html
editNoun.html
greeting.html
landing-page.html
NewUser.html
noun.html
take-test.html
test-result.html
application.properties
test/
java/
uk/
ac/
bangor/
cs/
group2/
academicymraeg/
target/
pom.xml
README.md
src/main/java/uk/ac/bangor/cs/group2/academicymraeg/
config/
FirstUserAutoConfigurer.java
NounInitialiser.java
SecurityConfig.java
UserDataInitialiser.java
repository/
NounRepository.java
TestQuestionRepository.java
TestRepository.java
UserRepository.java
controller/
AdminController.java
GreetingController.java
LoginController.java
NounViewController.java
TakeTestController.java
TestController.java
TestHistoryController.java
TestViewController.java
service/
DatabaseUserDetailsService.java
NounService.java
TestGeneratorService.java
UserService.java
models/
Noun.java
Question.java
Role.java
Test.java
TestQuestions.java
TestSubmissionForm.java
User.java
AcademiCymraegApplication.java
ServletInitializer.java
src/main/resources/
static/
images/
templates/
admin.html
editNoun.html
greeting.html
landing-page.html
NewUser.html
noun.html
take-test.html
test-result.html
application.properties
src/test/java/uk/ac/bangor/cs/group2/academicymraeg/
AcademiCymraegApplicationTests.java
NounSavingTest.java
NounViewTest.java
NounWordTest.java
controller/
AdminControllerTest.java
TakeTestControllerTest.java
TestControllerTest.java
service/
TestGeneratorServiceTest.java
Legend:
Folder
Java File
HTML File
Properties File
Word Document
Image File
PDF File
XML File
Markdown File

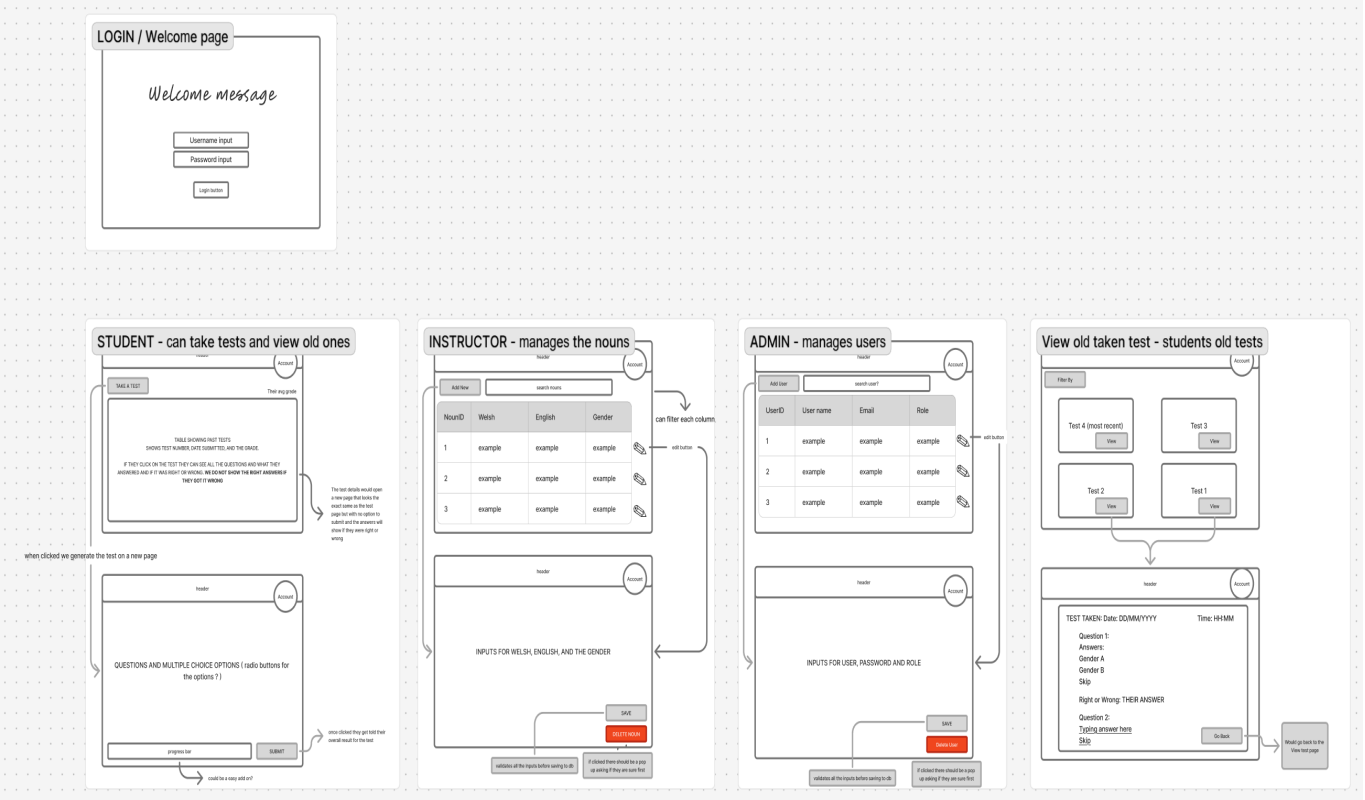
class diagram overview



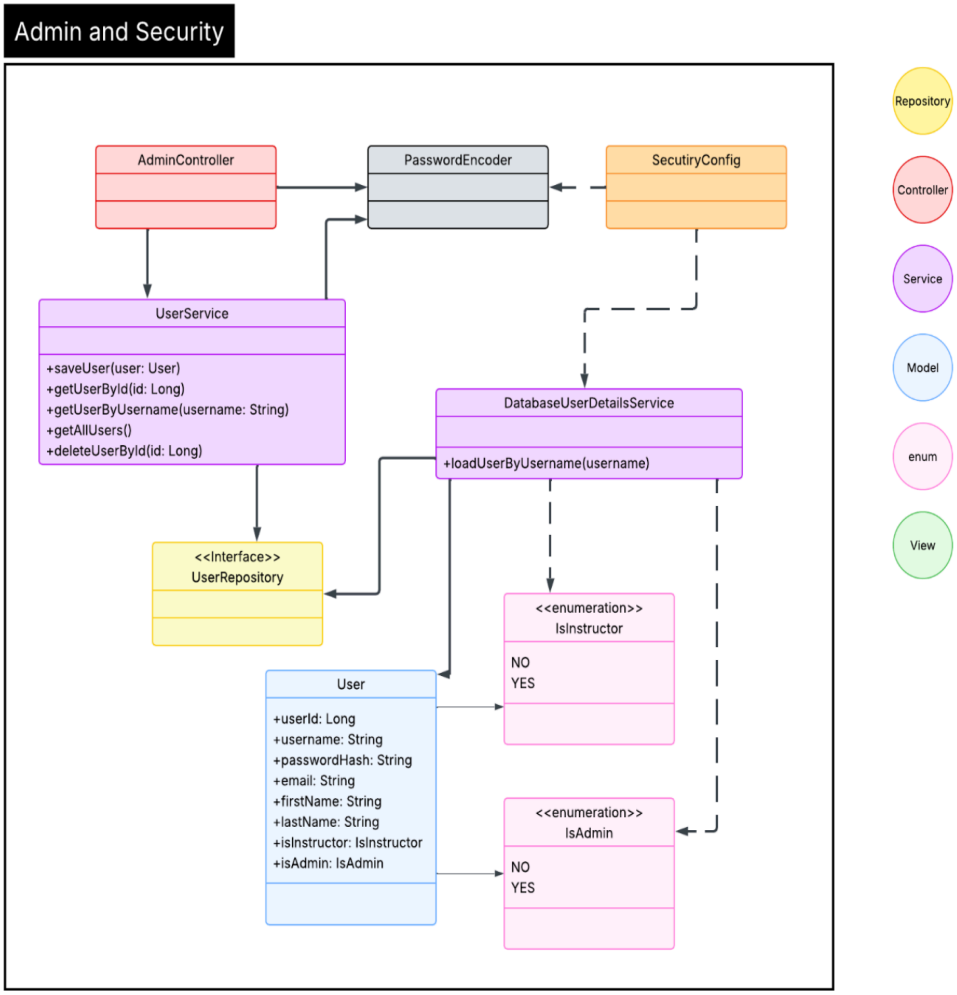
sequence diagram



screen wireframes



User Model



Defined in: models/User.java

Role control is handled via:

private IsInstructor isInstructor; private IsAdmin isAdmin;

User Persistence

Repository: repository/UserRepository.java

Key method: Optional findByUsername(String username);

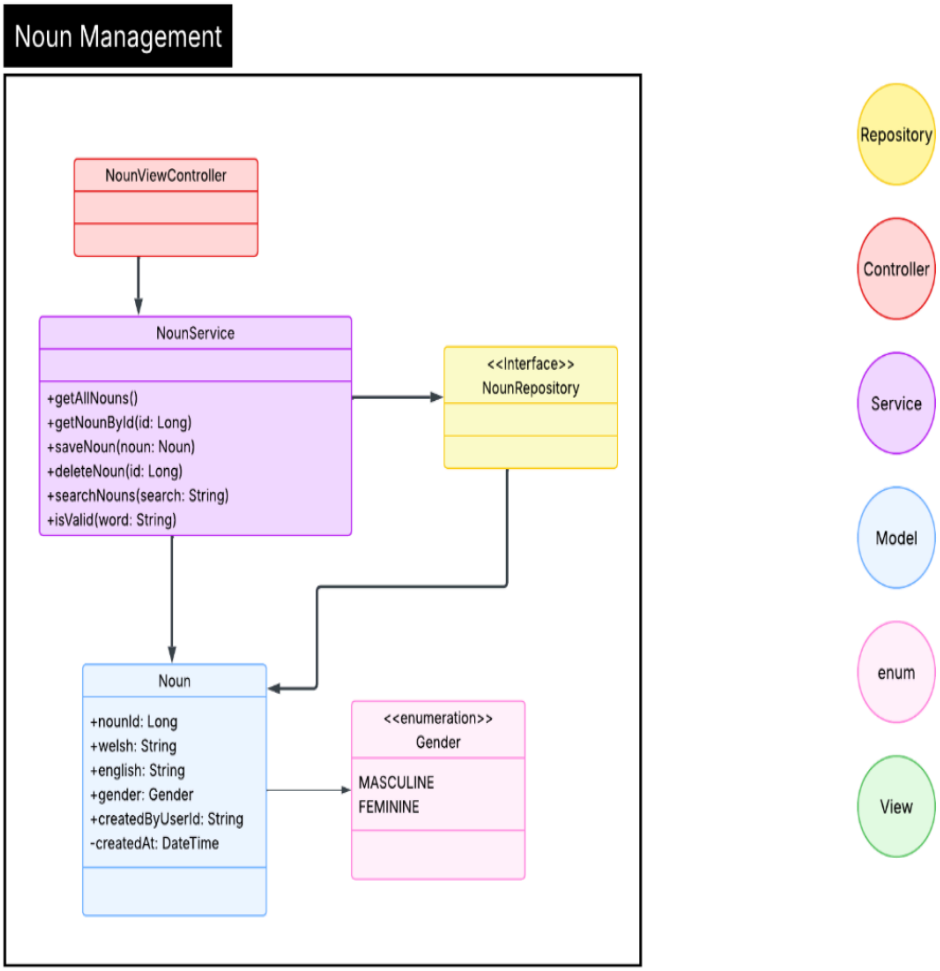
User Service

Defined in: service/UserService.java

Responsibilities:

- Create/update users
- Ensure passwords are BCrypt-encoded before saving
- Ensure role flags are not null

Nouns



Model: models/Noun.java

Fields:

- nounId
- english
- welsh
- gender (MASCULINE or FEMININE)
- createdByUsername
- createdAt

Repository: repository/NounRepository.java

Service: service/NounService.java

Responsibilities:

- Create new nouns
- Edit existing nouns
- Delete nouns

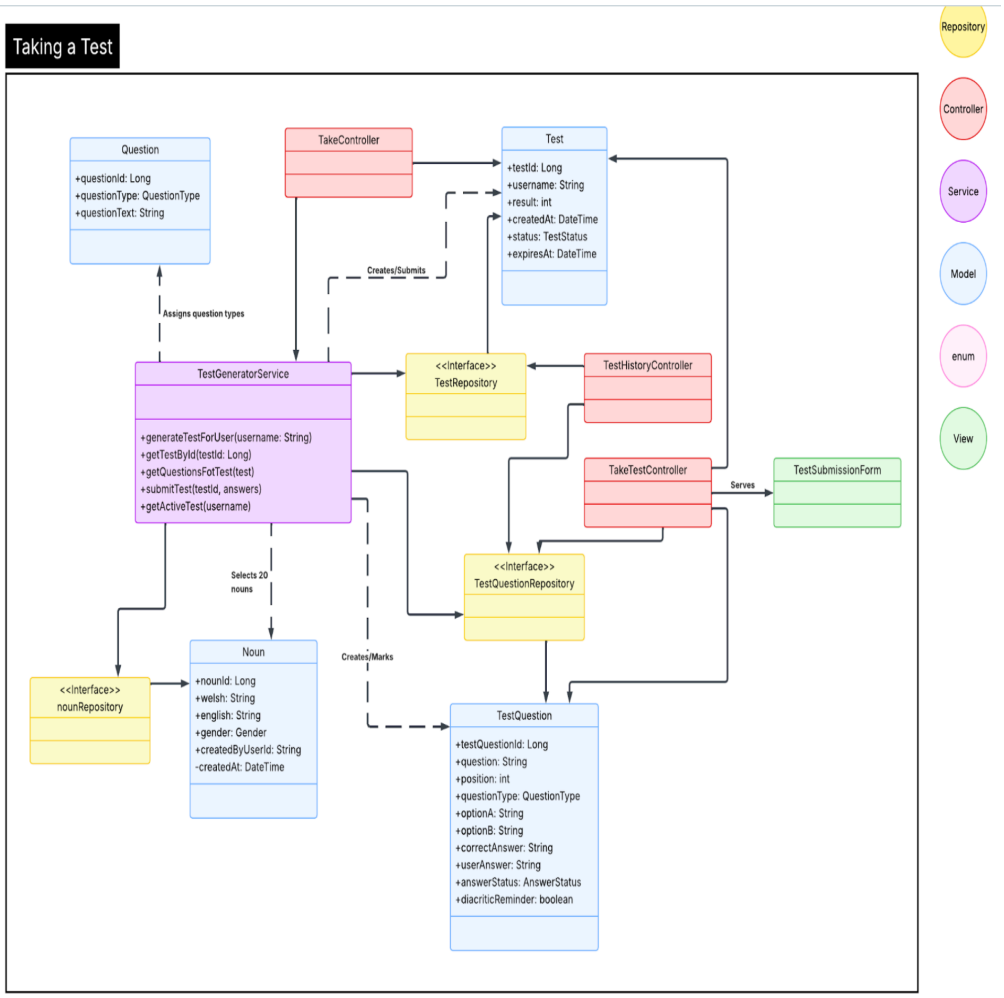
Access:

- INSTRUCTOR and SYSTEM_ADMIN only (/nouns/**)

Initial data: config/NounInitialiser.java

- Seeds 33 Welsh/English nouns on first run if the noun table is empty

Test Logic



Entities:

- models/Test.java (a single test taken by a user)
- models/TestQuestions.java (individual questions belonging to a test)
- models/Question.java (question type enum: WELSH, ENGLISH, GENDER)

Service: service/TestGeneratorService.java

Question types:

- WELSH → English to Welsh translation
- ENGLISH → Welsh to English translation
- GENDER → Gender of a Welsh noun (multiple choice)

Test composition:

- 20 questions per test
- 7 English to Welsh, 7 Welsh to English, 6 Gender
- Noun selection and question order are shuffled

Timer:

- 30 minute time limit per test
- On expiry any unanswered questions are marked SKIPPED and the test is auto-submitted

Answer checking:

- Exact match first
- Then a diacritic-tolerant check (e.g. "ty" will match "tŷ") with a reminder flag
- Gender questions require an exact MASCULINE or FEMININE answer

Anti-cheating:

- A user can only have one IN_PROGRESS test at a time
- Starting a test while one is in progress resumes the existing test
- Test history and review pages are blocked while a test is active (test-locked page)
- Browser caching of the test page is disabled to force a refresh on load

Persistence:

- Question text and correct answer are stored on TestQuestions
- Test results remain intact even if nouns or users are later deleted

Security Implementation

Authentication

- Implemented via UserDetailsService (DatabaseUserDetailsService)
- Users are loaded from the database using UserRepository
- Passwords are stored using BCrypt hashing

Authorization (Role-Based)

Roles are dynamically assigned based on flags in the User entity:

- ROLE_STUDENT (default for all users)
- ROLE_INSTRUCTOR (if isInstructor == YES)
- ROLE_SYSTEM_ADMIN (if isAdmin == YES)

Roles are constructed in: service/DatabaseUserDetailsService.java

Security Configuration

Defined in: config/SecurityConfig.java

Route Protection

/css/**, /images/**, /js/** → Public /tests/**, /my-tests/** → STUDENT, INSTRUCTOR, SYSTEM_ADMIN /nouns/** → INSTRUCTOR, SYSTEM_ADMIN /admin/**, /register/** → SYSTEM_ADMIN only /** → Authenticated users only

Spring automatically prefixes roles with ROLE_ Example: hasRole("SYSTEM_ADMIN") → ROLE_SYSTEM_ADMIN

Running the Application (Eclipse)

1. Import Project

- Open Eclipse
- File → Import
- Select "Existing Maven Project"
- Choose the project root

2. Build Project

- Right-click project → Maven → Update Project

3. Run Application

- Locate the main class annotated with @SpringBootApplication
- Right-click → Run As → Spring Boot App

4. Access Application http://localhost:8080

Database

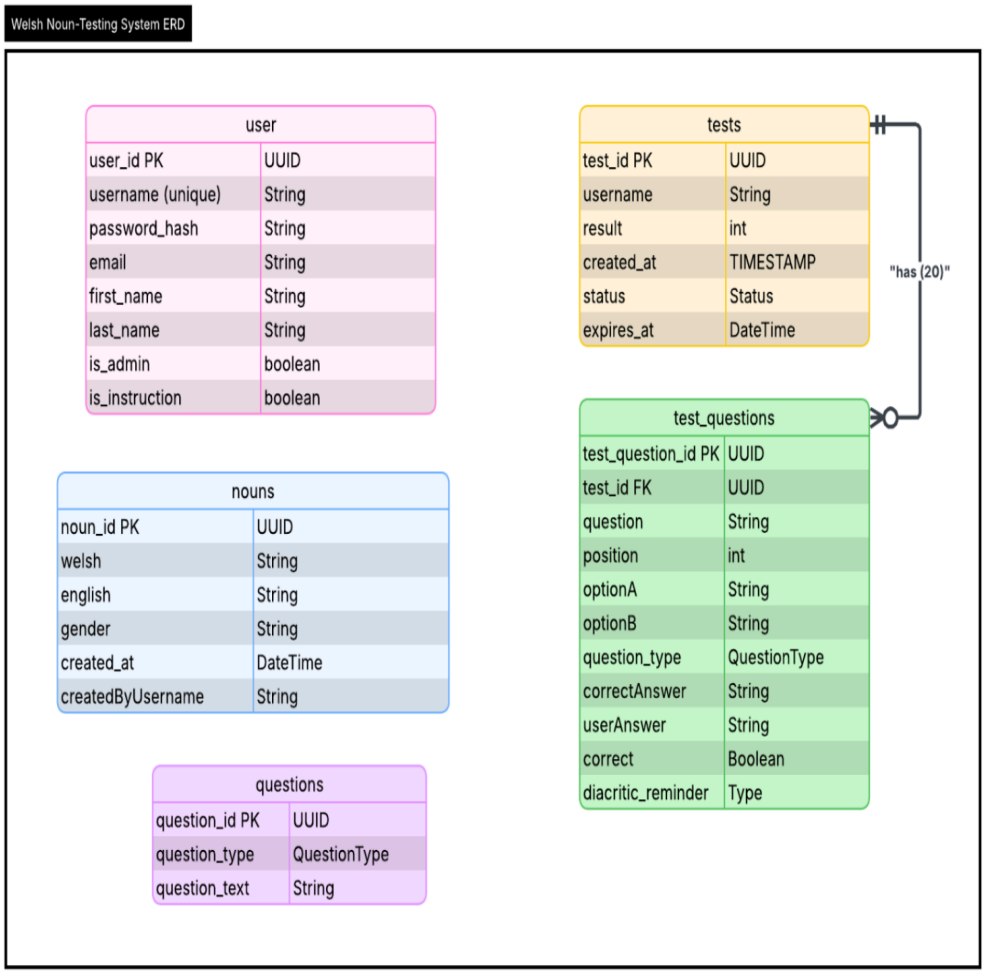
Configured in: src/main/resources/application.properties

Engine: MySQL (via mysql-connector-j)

Schema management: spring.jpa.hibernate.ddl-auto=update

Key tables:

- user (authentication details and role flags)
- noun (Welsh and English words with gender)
- test (test records per user)
- test_questions (questions belonging to a test)



Creating Users (Important)

Users are created in two ways:

Auto-created on startup: config/FirstUserAutoConfigurer.java

- Creates a default admin if one does not already exist
- Username: admin
- Password: password

Created by a System Administrator:

- Navigate to /register (SYSTEM_ADMIN only)
- Fill in the form to create a new user
- Edit or delete existing users from /admin

Known Constraints

- Role system is flag-based (no separate role table)
- All users receive STUDENT role by default
- CSRF is disabled (development only)

Development Notes

- Ensure controller routes match SecurityConfig

- Do not store plain-text passwords
- Do not enable in-memory users (overrides database authentication)
- Method-level security enabled via @EnableMethodSecurity